

Vitess

Vitess, YouTube ekibinin karşılaştığı MySQL ölçeklenebilirlik sorunlarını çözmek için 2010 yılında oluşturuldu. Bu bölüm, Vitess'in oluşturulmasına yol açan olaylar dizisini kısaca özetlemektedir:

1. YouTube, 'un MySQL veritabanı, en yüksek trafik seviyelerinin veritabanının hizmet kapasitesini aşacağı bir noktaya ulaştı. Sorunu geçici olarak hafifletmek için YouTube, yazma trafiği için birincil bir veritabanı (primary db – master db) ve okuma trafiği için (readonly db – slave db) bir çoğaltma (replica) veritabanı oluşturdu.
2. Kedi videolarına olan talep tüm zamanların en yüksek seviyesindeyken, salt okunur trafik bile çoğaltma veritabanını aşırı yükleyecek kadar yüksekti. Bu nedenle YouTube, geçici bir çözüm sağlamak için daha fazla çoğaltma ekledi.
3. Sonunda, yazma trafiği birincil veritabanının kaldırabileceğinden çok daha yüksek hale geldi ve YouTube'un gelen trafiği işlemek için verileri parçalara ayırması gerekti. Ayrıca, veritabanının genel boyutu tek bir MySQL örneği için çok büyük hale gelirse, parçalara ayırma da gerekli hale gelirdi.
4. YouTube'un uygulama katmanı, herhangi bir veritabanı işlemi yürütülmeden önce, kodun o belirli sorguyu alacak doğru veritabanı parçasını belirleyebilmesi için değiştirildi.

Vitess, YouTube'un bu mantığı uygulama kaynak kodundan kaldırmasına ve uygulama ile veritabanı arasında bir proxy yerleştirerek veritabanı etkileşimlerini yönlendirmesine ve yönetmesine olanak sağladı. O zamandan beri YouTube, kullanıcı tabanını 50 kattan fazla büyüttü ve sayfa sunma, yeni yüklenen videoları işleme ve daha fazlasını yapma kapasitesini büyük ölçüde artırdı. Daha da önemlisi, Vitess ölçeklenmeye devam eden bir platformdur.

Şubat 2018 de Teknik Gözetim Komitesi (Technical Oversight Committee - TOC), Vitess'i bir CNCF kuluçka (CNCF incubation) projesi olarak kabul etme kararı aldı. Vitess, Kasım 2019 da Kubernetes, Prometheus, Envoy, CoreDNS, containerd, Fluentd ve Jaeger'e katılarak CNCF'den mezun olan sekizinci proje oldu.

Vitess	1
1. Vitess Nedir?	2
1.1. Neden Vitess Kullanmalısınız?	3
1.2. Nasıl Çalışır?	3
1.3. Kullanım	3
1.4. Özellikler	4
1.4.1. Performans	4
1.4.2. Koruma (Protection)	4
1.4.3. İzleme (Monitoring)	4
1.4.4. Topoloji Yönetim Araçları (Topology Management Tools)	4
1.4.5. Parçalama (Sharding)	4
2. Vitess Mimarisi	5
3. Ölçeklenebilirlik Felsefesi	6
3.1. Küçük Örnekler	7
3.2. Replikasyon Yoluyla Kalıcılık	7
3.3. Tutarlılık Modeli (Consistency Model)	8
3.4. Aktif-Aktif Replication (Multi-Lead Replication) Desteklenmez	9
3.5. Multi-Cell (Çoklu Hücre)	10

1. Vitess Nedir?

Vitess, MySQL'in yatay ölçeklendirilmesi (horizontal scaling) için geliştirilmiş açık kaynaklı bir kümeleme sistemidir. Verileri parçalama yöntemiyle birden fazla MySQL örneğine dağıtırken, uygulamanıza tek bir birleşik veritabanı arayüzü sunar. Sorgular sanki tek bir MySQL sunucusuna gidiyormuş gibi çalışır, ancak Vitess bunları otomatik olarak ilgili parçalara (shard'lara) yönlendirir.

Vitess, genel ve özel bulut ortamlarında çalışır. MySQL ve Percona Server for MySQL'i destekler.

1.1. Neden Vitess Kullanmalısınız?

Vitess, MySQL ölçeklendirmesinde ortaya çıkan üç zorluğu ele alır:

1. **Tek Bir Sunucunun Ötesine Ölçeklendirme:** Vitess, uygulamanıza sharding mantığı eklemenizi gerektirmeden verileri birden fazla MySQL örneğine dağıtır.
2. **Veritabanı Kümelerini Yönetme:** Vitess, birçok MySQL örneğinde arıza durumlarını, yedeklemeleri ve topoloji değişikliklerini yönetir.
3. **Veritabanınızı Koruma:** Vitess, bağlantıları havuzlar, sorunlu sorguları yeniden yazar ve herhangi bir sorgunun performansı düşürmesini önlemek için sınırlar uygular.

1.2. Nasıl Çalışır?

Vitess, uygulamanız ve MySQL arasında yer alır. Şunlardan oluşur:

- **VTGate:** Uygulamanızdan MySQL protokol bağlantılarını kabul eden ve sorguları uygun parçalara yönlendiren bir proxy
- **VTTablet:** Her MySQL örneğinin yanında çalışan ve sorguları, bağlantıları ve replikasyonu yöneten bir ajan.
- **Topology Service:** Küme durumunu koruyan tutarlı bir veri deposu (etcd ZooKeeper veya Consul).

Uygulamanız, MySQL uyumlu herhangi bir sürücüyü (driver) kullanarak VTGate'e bağlanır. Vitess, optimize edilmiş performans için yerel JDBC ve Go sürücülerini de içerir.

1.3. Kullanım

Vitess, beş yıldan fazla bir süre boyunca tüm YouTube veritabanı trafiğine hizmet etti. Slack, Square ve JD.com gibi şirketler Vitess'i üretim ortamında kullanıyor.

1.4. Özellikler

1.4.1. Performans

- **Connection Pooling (Bağlantı Havuzlama):** Performansı optimize etmek için ön uç uygulama sorgularını bir MySQL bağlantı havuzuna çoklayın.
- **Query de-duping (Sorgu Tekilleştirme):** Halihazırda çalışmakta olan bir sorgunun sonuçlarını, o sorgu henüz yürütülmeye devam ederken gelen birebir aynı istekler için yeniden kullanılır.
- **Transaction Manager (İşlem Yöneticisi):** Genel verimliliği optimize etmek için eşzamanlı transaction sayısını sınırlayın ve zaman aşımı sürelerini yönetin.

1.4.2. Koruma (Protection)

- **Query Rewriting and Sanitization (Sorgu Yeniden Yazma ve Temizleme):** Sınırlar ekleyin ve deterministik olmayan güncellemelerden kaçınin.
- **Query Blocking (Sorgu Engelleme):** Potansiyel olarak sorunlu sorguların veritabanınıza ulaşmasını önlemek için kuralları özelleştirin.
- **Query Killing (Sorgu Öldürme-Sonlandırma):** Veri döndürmesi çok uzun süren sorguları sonlandırın.
- **Table ACL's (Table Access Control Lists):** Bağlı kullanıcıya göre tablolar için erişim kontrol listeleri (Access Control Lists- ACLs) belirtin.

1.4.3. İzleme (Monitoring)

Performans analiz araçları, veritabanı performansınızı izlemenize, teşhis etmenize ve analiz etmenize olanak tanır.

1.4.4. Topoloji Yönetim Araçları (Topology Management Tools)

- Küme yönetim araçları (Cluster Management Tools) planlı ve plansız arıza durumlarını yönetir.
- Web tabanlı yönetim arayüzü
- Birden fazla veri merkezi/bölgede çalışacak şekilde tasarlanmıştır.

1.4.5. Parçalama (Sharding)

- Neredeyse sorunsuz dinamik yeniden parçalama (re-sharding).
- Dikey ve yatay parçalama desteği.
- Özel olanları ekleyebilme özelliğiyle birden fazla parçalama şeması

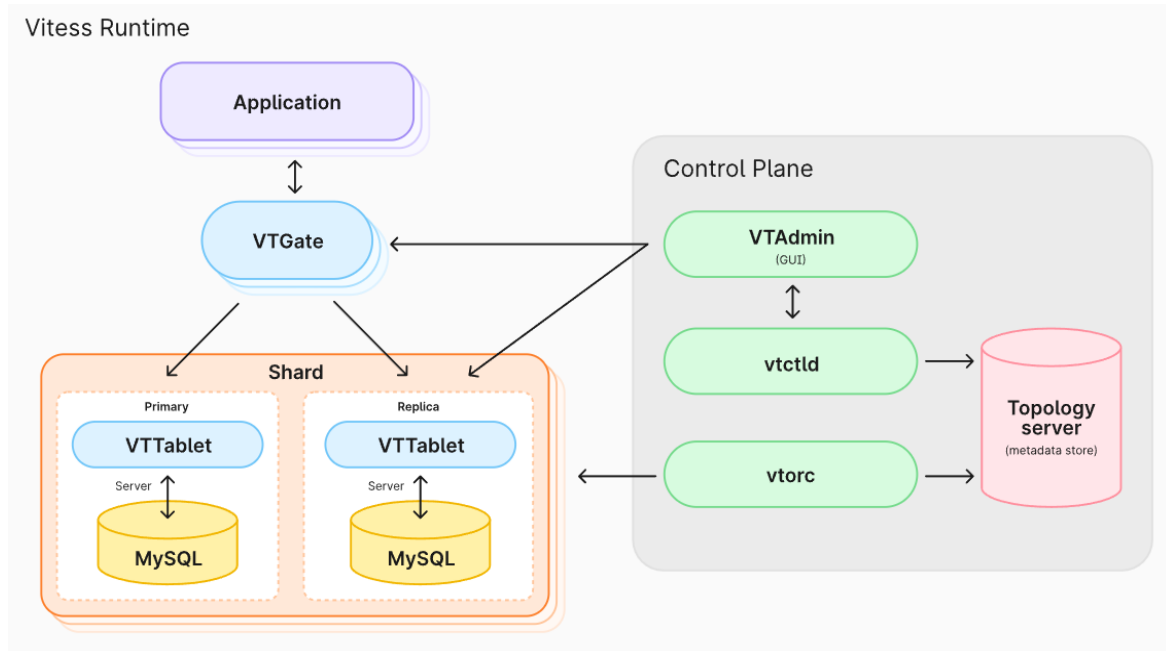
2. Vitess Mimarisi

Vitess platformu, tutarlı bir meta veri deposuyla (consistent metadata store) desteklenen bir dizi sunucu işlemi, komut satırı yardımcı programı ve web tabanlı yardımcı programdan oluşmaktadır.

Uygulamanızın mevcut durumuna bağlı olarak, tam bir Vitess kurulumuna farklı süreç akışlarını izleyerek ulaşabilirsiniz. Örneğin, sıfırdan bir servis inşa ediyorsanız, Vitess ile atacağınız ilk adım veritabanı topolojinizi (database topology) tanımlamak olacaktır. Ancak, mevcut bir veritabanını ölçeklendirmeniz gerekiyorsa, işe muhtemelen Vitess'i yönetilmeyen (Unmanaged) modda yayına alarak başlarsınız.

Vitess araçları ve sunucuları, ister devasa bir veritabanı filosuyla başlayın, ister küçük başlayıp zamanla ölçeklenin, size her durumda yardımcı olacak şekilde tasarlanmıştır. Daha küçük ölçekli durumlar için, VTablet'in bağlantı havuzlaması (connection pooling) ve sorguyu yeniden yazma (query rewriting) gibi özellikleri, mevcut donanımınızdan en yüksek verimi almanıza yardımcı olur. Vitess'in otomasyon araçları ise daha büyük ölçekli kurumlar için ek faydalar sağlar.

Şekil 2.1. Vitess'in bileşenlerini göstermektedir.



Şekil 2.1.

Şekil 2.1. de ki bileşenler Kavramlar kısmında incelenmiştir. (Bkz. “Kavramlar”)

3. Ölçeklenebilirlik Felsefesi

Ölçeklenebilirlik (Scalability) sorunları birçok yaklaşımla çözülebilir. Bu belge, Vitess'in bu sorunları ele alma yaklaşımını açıklamaktadır.

3.1. Küçük Örnekler

Veritabanlarını daha küçük parçalara ayırmaya veya bölmeye karar verirken, bunları tek bir makineye sığacak kadar bölmek cazip gelebilir. Sektörde, sunucu başına yalnızca bir MySQL örneği çalıştırmak yaygındır.

Vitess, örneklerini yönetilebilir parçalara (MySQL sunucusu başına 250 GB) bölünmesini ve sunucu başına birden fazla örnek çalıştırmaktan çekinmemeyi önerir. Net kaynak kullanımı yaklaşık olarak aynı olacaktır. Ancak MySQL örnekleri küçük olduğunda yönetilebilirlik (manageability) büyük ölçüde artar. Portları takip etme ve MySQL örnekleri için yolları ayırma karmaşıklığı vardır. Ancak bu engel aşıldıktan sonra her şey daha basit hale gelir.

Endişelenecek daha az kilit çekişmesi (lock contentions) vardır, replikasyon çok daha sorunsuz çalışır, kesintilerin üretim üzerindeki etkisi daha küçüktür, yedeklemeler ve geri yüklemeler daha hızlı çalışır ve daha birçok ikincil avantaj elde edilebilir. Örneğin, makine veya raf çeşitliliğini artırmak için örnekleri yeniden düzenleyerek kesintilerin üretim üzerindeki etkisini daha da azaltabilir ve kaynak kullanımını iyileştirebilirsiniz.

3.2. Replikasyon Yoluyla Kalıcılık

Geleneksel veri depolama yazılımları, verileri diske yazıldığı anda kalıcı olarak kabul ediyordu. Ancak bu yaklaşım, günümüzün standart donanım dünyasında pratik değildir. Bu yaklaşım ayrıca felaket senaryolarını da ele almaz.

Yeni kalıcılık yaklaşımı, verilerin birden fazla makineye ve hatta coğrafi konuma kopyalanmasıyla elde edilir. Bu kalıcılık biçimi, cihaz arızaları ve felaketler gibi modern endişeleri de ele alır.

Vitess'teki birçok iş akışı bu yaklaşım göz önünde bulundurularak oluşturulmuştur. Örneğin, yarı senkronize replikasyonun (semi-sync replication) açılması şiddetle tavsiye edilir. Bu, birincil sunucu (master db) çöktüğünde Vitess'in veri kaybı olmadan yeni bir

kopyaya geçmesini sağlar. Vitess ayrıca çökmüş bir veritabanını kurtarmaktan kaçınmanızı önerir. Bunun yerine, yakın tarihli bir yedeklemeden yeni bir veritabanı oluşturun ve güncellenmesini bekleyin.

Replikasyona güvenmek, disk tabanlı kalıcılık ayarlarının bazılarını gevşetmenize de olanak tanır. Örneğin, diske yapıla IOPS (Input Output Per Second – Saniyede Giriş Çıkış İşlemi) sayısını büyük ölçüde azaltarak verimi artıran sync_binlog'u kapatabilirsiniz.

3.3. Tutarlılık Modeli (Consistency Model)

Tabloları parçalamadan veya tabloları farklı anahtar alanlarına (keyspace) taşımadan önce uygulamanın aşağıdaki değişiklikleri tolere edebileceğinin doğrulanması (veya buna göre değiştirilmesi) gerekir.

- **Shard'lar arası okumalar (Cross-shard reads) birbiriyle tutarlı olmayabilir:** Sharding kararı verilirken bu tür durumların oluşması en aza indirilmeye çalışılmalıdır; çünkü shard'lar arası okumalar daha maliyetlidir.
- **En iyi çaba modunda (best effort mode), Shard'lar arası işlemler (cross-shard transactions) yarıda başarısız kalabilir veya kısmi işlem tamamlamalarına (partial commits) yol açabilir:** Bunun yerine, size dağıtık atomiklik garantileri sağlayan "2PC modu" transaction modelini kullanabilirsiniz. Ancak bu seçeneğin tercih edilmesi yazma maliyetini yaklaşık %50 oranında artırır.

Tek shard üzerindeki işlemler (Single shard transaction), tıpkı MySQL'in yerel olarak desteklediği gibi ACID kalmaya devam eder.

Biraz eski verilere dayanabilen salt okunur kod yolları vara, sorgular OLTP için REPLICA tabletlerine ve OLAP iş yükleri için RDONLY tabletlerine gönderilmelidir. Bu okuma trafiğinizi daha kolay ölçeklendirebilmenizi ve bunları coğrafi olarak dağıtmanızı sağlar.

Veri değiştikçe okumalar ana sunucunun (master db) gerisinde kalabileceğinden, bu ödünleşim muhtemelen tutarsız veri okumalarına yol açarken en iyi toplam verim sağlar. Replikasyon gecikmesini hafifletmek için VTGate sunucuları replika gecikmesini izleme yeteneğine sahiptir ve x saniyeden fazla geride kalan örneklerden veri sunmaktan kaçınacak şekilde yapılandırılabilir.

Yazma sonrası okuma tutarlılığı için, transaction olmadan birincil sunucudan okuma yeterlidir.

Özetle, çeşitli tutarlılık seviyeleri şunlardır:

- **REPLICA/RDONLY Read:** Sunucular coğrafi olarak ölçeklenebilir. Yerel okumalar hızlıdır, ancak çoğaltma gecikmesine (replication lag) bağlı olarak güncel olmayabilir.
- **PRIMARY Read:** Her shard için yalnızca dünya çapında bir adet birincil sunucu vardır. Uzak konumlardan gelen okumalar ağ gecikmesine ve güvenilirliğine tabi olacaktır, ancak veriler güncel olacaktır (read after write consistency) izolasyon seviyesi READ_COMMITTED'dir.
- **PRIMARY Transactions:** Bunlar primary okumalarla aynı özelliklere sahiptir. Ancak, tek bir shard için REPEATABLE_READ tutarlılığı ve ACID yazmaları elde edersiniz. Shard'lar arası atomik işlemler için destek planlanmaktadır.

Atomiklik (Atomicity) tarafında ise aşağıdaki seviyeler desteklenmektedir:

- **SINGLE:** Multi-db transaction'a izin vermez.
- **MULTI:** En iyi çaba taahhüdü ile multi-db transaction'larına izin verir.
- **TWOPC:** 2PC taahhüdü ile multi-db işlemlerine izin verir.

3.4. Aktif-Aktif Replication (Multi-Lead Replication) Desteklenmez

Vitess, aktif-aktif (eski adıyla multi-lead) replikasyon kurulumunu desteklemez. Tipik olarak bu tür bir yapılandırmayla çözülen kullanım senaryolarının çoğu için alternatif çözüm yollarına sahiptir.

- **Ölçeklenebilirlik (Scalability):** Aktif-aktif çoğaltmanın performans açısından ekstra zaman sağladığı durumlar. Ancak, tüm yazma işlemlerinin sonunda tüm yazılabilir sunuculara uygulanması gerektiğinden, sürdürülebilir bir strateji değildir. Vitess bu sorunu, süresiz olarak ölçeklenebilen parçalama (sharding) yoluyla çözer.
- **Yüksek Erişilebilirlik (High Availability):** Vitess, ana sunucu (primary) arızalarını tespit etme ve arıza tespitinden sonraki saniyeler içinde yeni bir primary sunucuya

geçiş (failover) yapma konusunda yerleşik bir yeteneğe sahiptir. Bu, çoğu uygulama için genellikle yeterlidir.

- **Coğrafi olarak dağıtık yazma işlemlerinde düşük gecikme süresi (Low-latency geographically distributed writes):** Bu, Vitess tarafından ele alınmayan bir durumdur. Mevcut öneri, yazma işlemleri için uzun mesafeli gidiş-dönüşlerin gecikme maliyetini üstlenmektir. Veri dağıtımı izin veriyorsa, coğrafi yakınlığa dayalı parçalama seçeneğiniz hala mevcuttur. Ardından, farklı coğrafi konumlarda bulunan farklı parçalar için birincil sunucular ayarlayabilirsiniz. Bu şekilde, birincil yazma işlemlerinin çoğu yine de yerel olabilir.

3.5. Multi-Cell (Çoklu Hücre)

Vitess, birden fazla veri merkezinde (data centers)/bölgede (region)/hücrede (cell) çalışacak şekilde tasarlanmıştır. Bu bölümde, “hücre (cell)” terimini birbirine çok yakın ve aynı bölgesel kullanılabilirliği paylaşan bir sunucu kümesi anlamında kullanacağız.

Bir hücre tipik olarak bir dizi tablet, bir vtgate pool ve Vitess kümesini kullanan uygulama sunucuları içerir. Vitess ile tüm bileşenler gerektiği gibi yapılandırılabilir ve çalıştırılabilir:

- Bir shard için birincil sunucu herhangi bir hücrede olabilir. Hücreler arası birincil sunucu erişimi gerekiyorsa, vtgate bunu kolayca yapacak şekilde yapılandırılabilir (birincil sunucuyu içeren hücreyi izlenecek hücre olarak geçirerek).
- Birincil sunucuyu içerebilen hücrelerin, salt okunur hizmet veren hücrelerden daha fazla kaynak ayrılmış olması yaygın bir durumdur. Bu birincil sunucuya sahip hücreler, aynı replika hizmet kapasitesini korurken olası bir arıza durumunda bir replika daha ihtiyaç duyabilir.
- Bir hücredeki birincil sunucudan farklı bir hücredeki birincil sunucuya geçiş, yerel bir arıza durumundan farklı değildir. Bu durum trafik ve gecikme üzerinde bir etkiye sahip olsa da uygulama trafiği de yeni hücreye yönlendirilirse, sonuç istikrarlı olur.
- Ayrıca, birincil sunucusu bir hücrede olan bazı shard'lar ve birincil sunucusu başka bir hücrede olan diğer shard'lar da olabilir. vtgate, trafiği doğru yere yönlendirecek ve yalnızca uzaktan erişimde ek gecikme maliyetine neden olacaktır. Örneğin, birincil sunucuları ABD'de olan bir veritabanında ABD kullanıcı kayıtlarını ve birincil sunucuları Avrupa'da olan bir veritabanında Avrupa kullanıcı

kayıtlarını oluşturmak kolaydır. Her hücrede replikalar bulunabilir ve replika trafiğini hızlı bir şekilde karşılayabilirler.

- Kullanıcı tarafından görülebilen gecikmeyi azaltmak için replika sunan hücreler iyi bir uzlaşmadır: yalnızca replika sunucuları içerirler ve birincil erişim her zaman uzaktan yapılır. Uygulama profili çoğunlukla okuma işlemleri içeriyorsa, bu gerçekten iyi çalışır.
- Tüm hücrelerin rdonly örneklerine ihtiyacı yoktur. Yalnızca, OLAP işleri çalıştıran hücrelerin bunlara gerçekten ihtiyacı vardır.

Vitess'in öncelikle yerel hücre verilerini kullandığını ve herhangi bir hücrenin devre dışı kalmasına karşı oldukça dayanıklı olduğunu unutmayın.

Vitess, veritabanlarının kapasitelerini kademeli olarak artırmasına olanak tanıdığı için bulut dağıtımları için oldukça uygundur. Vitess'i bulutta çalıştırmanın en kolay yolu Kubernetes üzerinden yapmaktır.

4. Kavramlar

Vitess'in temel kavramlarını ve terminolojisini öğrenin.

4.1. Cell (Hücre)

Bir Cell, bir alanda bir araya getirilmiş ve diğer Cell'lerdeki arızalardan izole edilmiş bir sunucu ve ağ altyapısı grubudur. Tipik olarak ya tam bir veri merkezidir (data center) ya da bir veri merkezinin bir alt kümesidir, bazen region (bölge) ya da availability region (kullanılabilirlik bölgesi) olarak da adlandırılır. Vitess, bir cell'in ağdan kesilmesi gibi cell düzeyindeki arızaları sorunsuz bir şekilde ele alır.

Bir Vitess uygulamasındaki her cell'in, o hücrede barındırılan local topology service'i (local topology hizmeti) vardır. Topology service'i, kendi cell'indeki Vitess tabletleri hakkındaki bilgilerin çoğunu içerir. Bu, bir cell'in tek bir birim olarak devre dışı bırakılmasını ve bir birim olarak yeniden oluşturulmasını sağlar.

Vitess, hem veri hem de meta veriler için hücreler arası trafiği sınırlandırır. Okuma trafiğini bireysel hücrelere yönlendirme yeteneğine sahip olmak da yararlı olsa da Vitess, şu anda okuma işlemlerini yalnızca yerel cell'lerden gerçekleştirir. Yazma işlemleri gerektiğinde o shard'ın birincil sunucusunun bulunduğu yere hücreler arası olarak gider.

4.2. Execution Plans (Yürütme Planları)

Vitess, bir sorguyu yürütmek için en iyi yöntemi değerlendirmek amacıyla sorguları hem VTGate hem de VTablet katmanında ayrıştırır (parse eder). Bu değerlendirme, sorgu planlaması (query planning) olarak bilinir ve bir sorgu yürütme planı (query execution plan) ile sonuçlanır.

Yürütme planı hem sorguya hem de onunla ilişkili olan VSchema'ya bağlıdır. Vitess'in planlama stratejisinin temel amaçlarından biri, mümkün olduğunca fazla işi alttaki MySQL instance'larına yıkmaktır (push down). Bunun mümkün olmadığı durumlarda Vitess, birden fazla kaynaktan girdi toplayan ve doğru sorgu sonucunu üretmek için bu sonuçları birleştiren (merge) bir plan kullanacaktır.

- **Push Down (Aşağı Yıkmak/İtmek):** Vitess, gelen SQL sorgusunu kendisi işleyip CPU harcamak istemez. Eğer sorgu tek bir shard'ın içindeki verilere bakıyorsa,

Vitess sorguyu hiç bozmadan doğrudan o shard'daki MySQL'e gönderir. İşin yükünü tamamen MySQL'e yıkar. En performanslı senaryo budur.

- **Scatter and Merge (Dağıt ve Birleştir):** Eğer atılan sorgu birden fazla shard da yayılmış verileri kapsıyorsa (örneğin tüm shardlar'daki kullanıcılar kapsayan bir COUNT (*) sorgusu), VTGate bu işi MySQL'e bırakmaz. Planlama aşamasında sorguyu parçalar, tüm shard'lara aynı anda dağıtır (scatter), her shard'dan gelen ara sonuçları kendi hafızasında birleştirir (merge) ve nihai sonucu üretir.

4.2.1. Evaluation Model (Değerlendirme Modeli)

Bir yürütme planı, her biri belirli bir iş parçasını uygulayan operatörlerden (operators) oluşur. Operatörler, genel yürütme planını temsil eden ağaç benzeri bir yapıda birleşir. Plan, her operatörü ağaçtaki bir düğüm olarak temsil eder. Her operatör, sıfır veya daha fazla satır girdisi alır ve sıfır veya daha fazla satır çıktısı üretir. Bu, bir operatörden gelen çıktının bir sonraki operatör için girdi haline geldiği anlamına gelir. Ağaçtaki iki dalı birleştiren operatörler, iki akıştan gelen girdileri birleştirir ve tek bir çıktı üretir.

Özetle; Sorgunun boyutu ne olursa olsun, Vitess'in o sorguyu çalıştırmak için arka planda oluşturduğu bütünsel yol haritasına "Plan" denir. Bu planın içindeki her bir küçük iş adımına (filtreleme, yönlendirme, birleştirme) ise "Operatör" denir, yani plan:

- **Tek bir operatörden oluşan planlar olabilir:** Örneğin çok basit bir nokta atışı sorgu attın (SELECT * FROM Users WHERE Id = 5). Vitess bunu çözer, sadece tek bir yönlendirme operatörü (Route Operator) oluşturur ve doğrudan MySQL'e paslar. Burada **1 operatör = 1 plan** olur.
- **Onlarca operatörden oluşan planlar olabilir:** Eğer karmaşık, JOIN içeren, sıralama (ORDER BY) ve gruplama (GROUP BY) barındıran bir sorgu yazdıysan; ağaçta 2 tane yaprak düğüm operatörü, 1 tane join operatörü, 1 tane filter operatörü, 1 tane sort operatörü derken **10-15 operatör tek bir planı** oluşturabilir.

Yürütme planının değerlendirilmesi ağacın yaprak düğümlerinde başlar. Yaprak düğümler, VTTablet'ten, Topology Service'inden veri çeker ve bazı durumlarda ifade değerlerini yerel olarak da değerlendirebilir. Her yaprak düğüm, diğer diğer operatörlerden girdi almaz ve ürettikleri düğümleri üst düğümlere iletir. Üst düğümler

daha sonra düğümleri kendi üst düğümlerine iletir ve bu işlem kök düğüme kadar devam eder. Kök düğüm, sorgunun nihai sonuçlarını üretir ve sonuçları kullanıcıya iletir.

4.2.2. Routing Operators (Yönlendirme Operatörleri)

Bir yürütme planındaki yönlendirme operatörü, Vitess'e bir iş parçasını hangi hedefe göndereceğini bildirir. Tipik olarak bir yönlendirme operatörü Vitess'a; iş parçası yürütülürken hangi keyspace'in kullanılacağını, bu keyspace'in shard olup olmadığını ve shard edilmiş ise hangi vindex'in kullanılacağını söyler.

4.2.3. Scatter Queries (Dağıtma Sorguları)

Shard edilmiş bir keyspace belirten ancak bir vindex belirtmeyen yönlendirme operatörü, shard edilmiş keyspace'deki tüm shard'lara "dağıtım (scatter)" yapacaktır. Bir scatter sorgusu, shard edilmiş bir keyspace'e yönlendirilen ancak bir vindex kullanılarak yönlendirilmeyen bir veya daha fazla iş parçası içerir.

Shard edilmiş bir keyspace'deki birden fazla shard'a gönderilen her sorgunun bir scatter sorgusu olarak kabul edilmediğini unutmayın.

Özetle: indeksiz (vindex belirtilmeyen) sorgu scatter olur, yani bütün shard'lara dağıtılır. İndeksli (vindex belirtilen) sorgu, scatter olmaz, hedef shard'ları bilir ve sadece sorguyu onlara dağıtır.

4.2.3. Observing Execution Plans (Yürütme Planlarını Gözleme)

Önbelleğe alınmış (cached) yürütme planları, VTGate düzeyinde /queryz uç noktasına (endpoint) göz atılarak incelenebilir.